

판다스 (pandas)

한국폴리텍대학 성남캠퍼스 인공지능소프트웨어과
이혜정 교수

1

Pandas의 개념

■ 데이터를 조작하고 분석하는 데 사용하는 파이썬 라이브러리 (파이썬계의 엑셀)

- 데이터를 표(테이블) 형태로 분석할 수 있음

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	weight_0
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296.0	15.3	396.9	4.98	1
1	0.02731	0.0	7.07	0	0.469	6.421	48.9	4.9671	2	242.0	17.8	396.9	9.14	1
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.83	4.03	1
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.63	2.94	1
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.9	5.33	1

- <https://pandas.pydata.org/docs/reference/index.html>

■ 사용

- Pandas 모듈 설치

```
conda install pandas
```

- Pandas 사용

- Pd라는 별칭 사용

```
import pandas as pd
```

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이혜정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

2

Pandas의 데이터 구조

- 데이터프레임(DataFrame) : 데이터 테이블 전체 객체
- 시리즈(Series) : 각 열 데이터를 다루는 객체

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	weight_0
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296.0	15.3	396.9	4.98	1
1	0.02731	0.0	7.07	0	0.469	6.421	48.9	4.9671	2	242.0	17.8	396.9	9.14	1
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.83	4.03	1
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.63	2.94	1
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.9	5.33	1

시리즈(Series)

데이터프레임 중 하나의 열에
해당하는 데이터의 모음 객체

데이터프레임(DataFrame)

데이터 테이블 전체를
포함하는 객체

※ 본 자료는 2026년 2학기 『시 기반 데이터 분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

3

Pandas의 데이터 구조

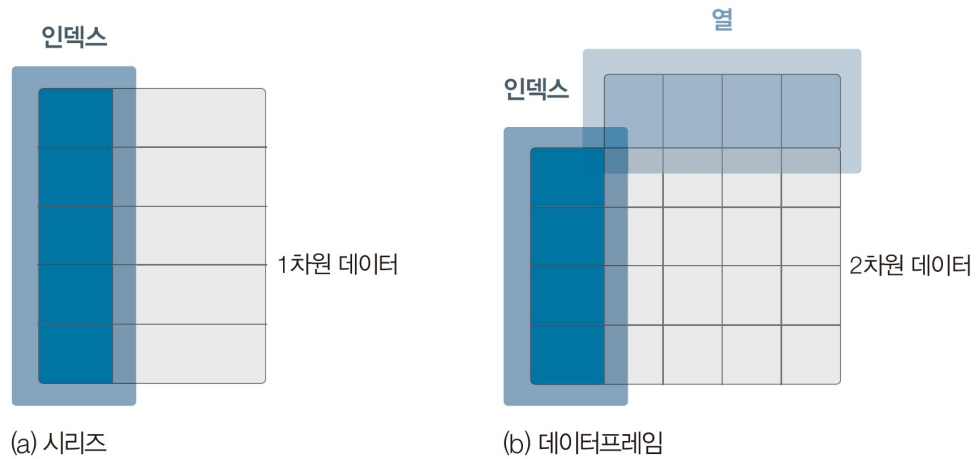
시리즈	• 1차원 데이터 • 리스트와 유사하나 리스트와 달리 인덱스를 부여할 수 있음 • sr 별칭을 많이 사용
데이터프레임	• 2차원 데이터 • 각각 인덱스를 가진 행(인덱스로 구분)과 열(열 이름으로 구분)로 이루어진 표 형태로 저장 • df 별칭을 많이 사용

※ 본 자료는 2026년 2학기 『시 기반 데이터 분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

4

Pandas의 데이터 구조

시리즈와 데이터프레임 구조



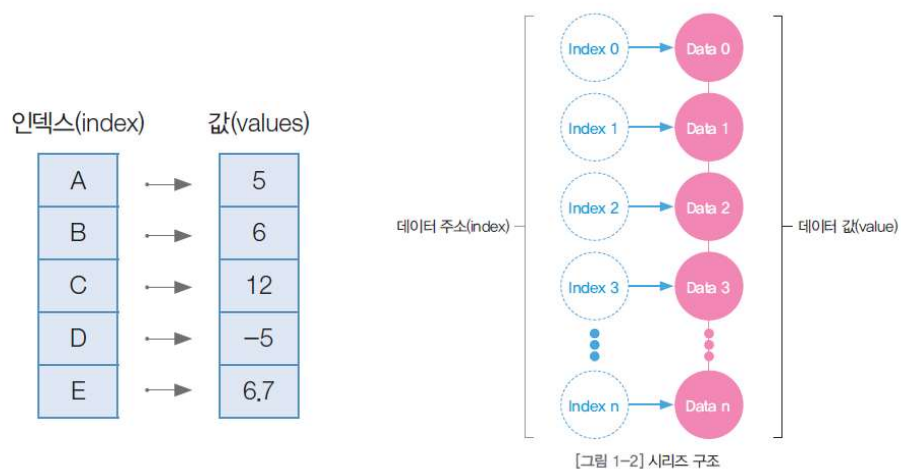
※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

5

시리즈 (Series)

데이터가 순차적으로 나열된 1차원 배열의 형태를 가짐

- List와는 달리 index를 가지고 있음



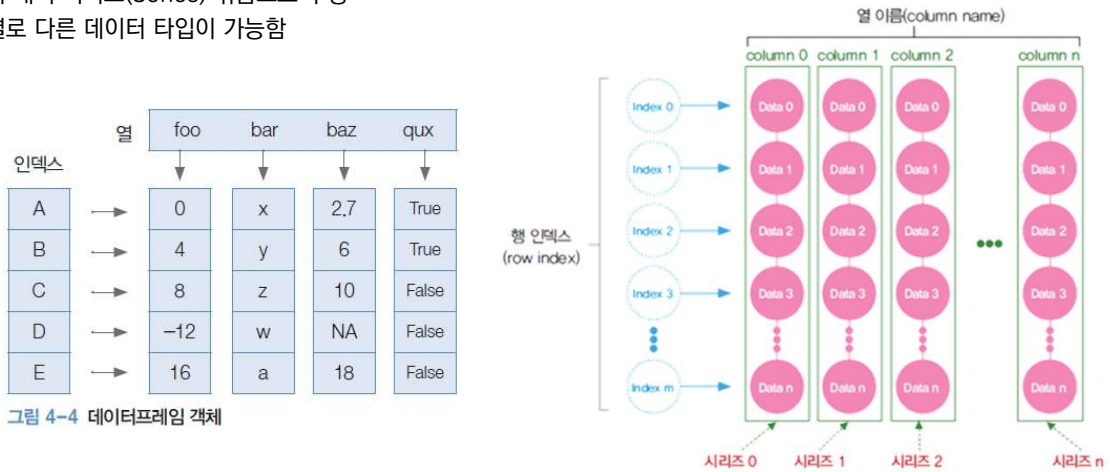
※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

6

데이터프레임 (Dataframe)

■ 데이터 테이블 전체를 지칭하는 객체

- 각각 인덱스를 가진 행 (index로 구분)과 열(이름으로 구분)로 이루어진 표 형태로 저장
- 여러 개의 시리즈(Series) 묶음으로 구성
- 열별로 다른 데이터 타입이 가능함



※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

판다스와 넘파이

■ 판다스는 데이터를 다루기 위해 넘파이(Numpy)를 기본적으로 사용

- 넘파이 배열 형태로 데이터 저장
- 넘파이의 데이터 타입을 기반으로 데이터 타입 정의
 - 판다스의 수치형 타입은 넘파이의 타입을 그대로 사용

데이터 종류	넘파이(NumPy) 타입	판다스(Pandas) 타입	설명
정수	int64, int32 등	int64	음수와 양수를 포함하는 정수
실수	float64, float32 등	float64	소수점이 있는 숫자
논리	bool_	bool	True 또는 False
객체/문자열	string_, unicode_	object	문자열 또는 여러 타입이 섞인 경우
시간	datetime64[ns]	datetime64[ns]	나노초 단위까지 표현하는 날짜/시간
시차	timedelta64[ns]	timedelta64[ns]	두 날짜 사이의 간격
범주	(없음)	category	고정된 종류의 데이터 (성별, 등급 등)

- 내부적으로 넘파이 연산을 호출하여 For문과 같은 loop 없이 한 번에 계산
 - 넘파이의 브로드캐스팅(broadcasting) 기능 사용
- 넘파이 수학 함수를 그대로 사용 가능

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

판다스의 인덱스 (index)

■ 특징

- 중복 허용
 - 판다스의 인덱스는 중복된 값을 가질 수 있음 (권장하지 않음)
- 다양한 타입
 - 정수 뿐만 아니라 문자열, 날짜(datetime) 등을 인덱스로 사용할 수 있음
- 불변성 (Immutability)
 - 인덱스 객체의 값은 직접 개별적으로 수정 불가 → 전체를 교체해야 함

■ 유형

- 정수형 위치 인덱스 (Range Index)
 - 따로 지정하지 않으면 0부터 시작하는 숫자가 자동으로 할당됨
- 레이블 인덱스 (Label Index)
 - 사용자가 의미 있는 이름을 직접 부여한 인덱스
- 시계열 인덱스 (Datetime Index)
 - 날짜나 시간 데이터를 인덱스로 사용 → 시계열 분석에 최적화

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

9

시리즈(Series) 객체의 생성

■ 리스트, 딕셔너리 등 시퀀스형 데이터로부터 생성

- 데이터프레임 객체를 먼저 생성하고 각 열에서 시리즈 객체를 뽑는 것이 일반적임

```
series = pd.Series(sequence, dtype=, index=, name=)
```

```
# 리스트로부터 시리즈 객체 생성
numbers = [1, 2, 3, 4, 5]
list_sr = pd.Series(numbers)
print(list_sr)
type(list_sr)

# 시리즈 객체 속성 확인
print(list_sr.dtype, list_sr.shape, list_sr.size, list_sr.name)
print(list_sr.index)
print(list_sr.values)

# 딕셔너리로부터 시리즈 객체 생성
dict_data = {"a":1, "b":2, "c":3, "d":4, "e":5}
dict_sr = pd.Series(dict_data)
print(dict_sr.dtype, dict_sr.shape, dict_sr.size, dict_sr.name)
print(dict_sr.index)
print(dict_sr.values)
```

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

10

10

시리즈(Series) 객체의 속성

속성명	설명
s.dtype	데이터 타입 반환
s.shape	데이터 구조(모양)를 튜플로 반환
s.size	요소의 총 개수 반환
s.name	시리즈의 이름 반환
s.index	시리즈의 인덱스 반환
s.values	시리즈의 값을 넘파이 배열로 반환

```
# 시리즈 객체 속성 설정 : dtype
sr1 = pd.Series(numbers, dtype=float)
print(sr1)
import numpy as np
sr2 = pd.Series(numbers, dtype=np.float32)
print(sr2)

# 시리즈 객체 속성 설정 : index, name
chars = ['a', 'b', 'c', 'd', 'e']
sr3 = pd.Series(numbers, index=chars, name='MySeries')
print(sr3.dtype, sr3.shape, sr3.size, sr3.name)
print(sr3.index)
print(sr3.values)

# 시리즈 객체 속성 수정/추가 : index, name
sr1.index = chars
sr1.name = 'MySeries'
print(sr1)
```

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

11

11

데이터프레임 객체의 생성과 속성 수정

- 리스트, 딕셔너리 등의 데이터로부터 데이터프레임 객체 생성

```
dataframe = pd.DataFrame(data, columns=, index=, ...)
```

속성	설명
df.shape	행과 열의 개수
df.columns	열 이름(컬럼명) 목록
df.index	행 인덱스 범위
df.dtypes	각 열의 데이터 타입
df.values	실제 값 (NumPy 배열)

- 컬럼명 수정 `new_df = org_df.rename(columns={org_col_name : new_col_name, ...})`
- 열의 데이터 타입 수정 `new_df = org_df.astype(columns={col_name : data_type, ...})`

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

12

12

데이터프레임 객체의 생성과 속성 수정

```
# 1차원 리스트로부터 데이터프레임 생성
numbers = [1, 2, 3, 4, 5]
pd1 = pd.DataFrame(numbers)
print(pd1)
pd2 = pd.DataFrame(numbers, columns=['number'])
print(pd2)

# 2차원 리스트로부터 데이터프레임 생성
score=[['정약용', 70, 80, 100],
        ['이순신', 100, 80, 70],
        ['이율곡', 90, 95, 90]]

score_df = pd.DataFrame(score)
score_df.columns = ['이름', '국어', '영어', '수학']
score_df2 = pd.DataFrame(score, columns = ['이름', '국어', '영어', '수학'])
```

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

13

13

데이터프레임 객체의 생성과 속성 수정

```
# 데이터프레임의 속성 확인
print(score_df.shape)
print(score_df.index)
print(score_df.columns)
print(score_df.dtypes)
print(score_df.values)

# 딕셔너리로부터 데이터프레임 생성
score_dict={'이름': ['정약용', '이순신', '이율곡'],
            '국어' : [70, 100, 90],
            '영어' : [80, 80, 95],
            '수학' : [100, 70, 90]}
score_df4 = pd.DataFrame(score_dict)
```

```
# 컬럼명 수정
score_df.rename(columns={'이름': 'name'})

# 컬럼 데이터의 자료형 수정
score_df.astype({'국어' :float})
```

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

14

14

데이터프레임의 열(칼럼) 추가

`dataframe[new_col_name] = data` 또는 `dataframe` 컬럼 간 연산

```
score_dict={'이름': ['정약용', '이순신', '이율곡', '홍길동'],
            '국어' : [70, 100, 90, 85],
            '영어' : [80, 80, 95, 100],
            '수학' : [100, 70, 90, 90]}
score_df = pd.DataFrame(score_dict)

score_df['사회'] = [85, 88, 92, 96]
score_df['총점'] = score_df['국어'] + score_df['영어'] + score_df['수학'] + score_df['사회']
score_df['평균'] = score_df['총점'] / 4
```

```
x = [0, 1, 2, 3, 4, 5]
y = [0, 2, 4, 6, 8, 10]
df = pd.DataFrame( )
df['x'] = x
df['y'] = y
df['x * y'] = df['x'] * df['y']
```

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

15

15

데이터프레임의 행/열 삭제, 인덱스 초기화

▪ drop 함수를 사용하여 행 또는 열을 삭제

- 삭제할 행의 index 또는 열이름을 명시하고 axis 속성으로 행 또는 열을 설정하여 사용

```
new_df = org_df.drop(list_of_index, axis=0, inplace=)
new_df = org_df.drop(list_of_col_name, axis=1, inplace=)
```

- axis가 없으면 행으로 처리

▪ reset_index 함수를 사용하여 정수 인덱스로 초기화

```
new_df = org_df.reset_index(drop=False, inplace=False)

score_df = pd.DataFrame(score, columns = ['이름', '국어', '영어', '수학'])
new_df1 = score_df.drop(['이름'], axis=1)
score_df.drop([0, 2], axis=0)
score_df.drop([0, 2])

score_df.reset_index(drop = True)
```

※ 본 자료는 2026년 2학기 『AI기반데이터분석 (이해정 교수)』 강의자료로 무단 배포 및 발해를 금합니다.

16

16